

Perfect — let's turn this into a **real structured course** 🎓

Not just explanations, but something that takes you from **zero** → **building your own document intelligence model (MinerU-level + better reasoning)**.

Course: *Advanced Embeddings & Document Intelligence Systems*

Course Goal

By the end, you will:

PDF → Nodes → Embeddings → Relations → Semantic Tree

AND understand **how models like LayoutLMv3 and MonkeyOCR actually work internally**

MODULE 1 — Foundations (You already started)

♦ **1.1 Why Transformers replaced RNNs**

From Attention Is All You Need

- RNN problem:
 - sequential bottleneck
 - long-range dependency loss
 - Transformer solution:
 - parallel attention
 - global context
-

♦ **1.2 Embeddings (Deep view)**

You learned:

Embedding = Token + Positional

Now upgrade it:

Embedding = Information Compression Function

👉 It encodes:

- meaning
 - position
 - relationships (implicitly)
-

♦ 1.3 Positional Encoding (REAL intuition)

Why sin/cos?

👉 Because:

PE(pos + k) can be expressed from PE(pos)

➡ That's how attention learns **relative distance**

MODULE 2 — Attention (Core of reasoning)

♦ 2.1 Self-attention intuition

Forget formula. Think:

Each token asks:

"Who should I look at?"

♦ 2.2 Attention = similarity engine

$\text{score}(i, j) = \text{similarity}(i, j)$

♦ 2.3 Why this matters for YOU

In documents:

Paragraph → attends to → Title

👉 That's already parent-child detection

MODULE 3 — From Text to Documents

Now we shift from NLP → Document AI

♦ 3.1 Problem shift

Text world:

1D sequence

Document world:

2D + hierarchy

♦ 3.2 Node representation

Each node:

```
{  
  "text": "...",  
  "bbox": [x1, y1, x2, y2]
```

}

◆ 3.3 Your NEW embedding

$E(\text{node}) =$
 TextEmbedding
 + SpatialEmbedding
 + StyleEmbedding



MODULE 4 — Spatial Embeddings (CRITICAL)

◆ 4.1 Why positional encoding fails

Because:

$\sin(\text{pos}) \rightarrow$ assumes sequence

But you need:

(x, y) space

◆ 4.2 Solution: 2D encoding

Used in LayoutLM:

$E_{\text{bbox}} = \text{embedding}(x1) + \text{embedding}(y1) + \text{embedding}(x2) + \text{embedding}(y2)$

◆ 4.3 Relative position (VERY IMPORTANT)

$$dx = x_i - x_j$$
$$dy = y_i - y_j$$

👉 This encodes:

- above / below
- left / right
- alignment

MODULE 5 — Relationship Modeling (THE CORE PROBLEM)

♦ 5.1 Your actual task

Given nodes → predict parent

♦ 5.2 Method 1 — Pairwise scoring

$$\text{score}(i, j) = \text{MLP}(E_i, E_j)$$

♦ 5.3 Method 2 — Attention (better)

$$\text{score}(i, j) = Q_i \cdot K_j$$

👉 Already inside transformers

♦ 5.4 Method 3 — Graph modeling (SOTA)

Document = Graph

Nodes = blocks

Edges = relations

Used in:

- Graphormer
-



MODULE 6 — Building the Tree

◆ 6.1 Parent prediction

$\text{parent}(i) = \text{argmax}_j \text{score}(i, j)$

◆ 6.2 Constraints

You must enforce:

- no cycles
 - one parent per node
 - top-down structure
-

◆ 6.3 Optional: beam search

Used to improve structure:

keep top-k tree candidates



MODULE 7 — Real Systems

◆ 7.1 LayoutLMv3

- text + layout + image
 - transformer-based
 - strong semantic understanding
-

◆ 7.2 MonkeyOCR

- OCR + structure parsing
 - outputs markdown / tree
 - uses learned relations
-

◆ 7.3 Your observation (VERY GOOD)

“MinerU extracts layout but doesn’t reason”

✅ Correct

👉 It lacks **deep relational modeling**



MODULE 8 — Training Without Data

◆ 8.1 Weak supervision

MonkeyOCR output → pseudo labels

◆ 8.2 Build dataset

(node_i, node_j, label)

◆ 8.3 Train:

MLP or Transformer

MODULE 9 — Advanced Embeddings (NEXT LEVEL)

◆ 9.1 Pair embedding (your idea)

[CLS] node_i [SEP] node_j

👉 This is a **cross-encoder**

◆ 9.2 Better formulation

$E_{\text{pair}} =$
 E_i
 + E_j
 + $(E_i - E_j)$
 + spatial_diff

◆ 9.3 Why it works

Because:

relationships $\neq$ individual meaning

MODULE 10 — Your Final System

Architecture:

PDF



MinerU (layout nodes)



Node Encoder (text + bbox)



Relation Model (attention / MLP)



Tree Builder



Semantic Tree 



PRACTICAL PROJECT (IMPORTANT)

Build this:

Step 1

Extract nodes from PDF

Step 2

Create embeddings

Step 3

Train:

predict parent(node)

Step 4

Visualize tree

What makes you advanced after this

You'll understand:

- ✓ embeddings beyond NLP
 - ✓ attention as relation modeling
 - ✓ document structure learning
 - ✓ how SOTA systems actually work
-

If you want next level

I can turn this into:

-  hands-on coding course (PyTorch)
 -  math deep dive (attention proofs)
 -  build your own MinerU competitor
-

Just say:

“let's build module 1 code”

or

“go deeper into attention math”

and we go step-by-step like a real course 🔥